Kazakh-British Technical University

Data collection and preparation

STUDENTS' INDEPENDENT STUDY (SIS) №1

Manga Sales vs Critical Reception

Full Name: Nursultan Zhantuar

ID: 23B030356

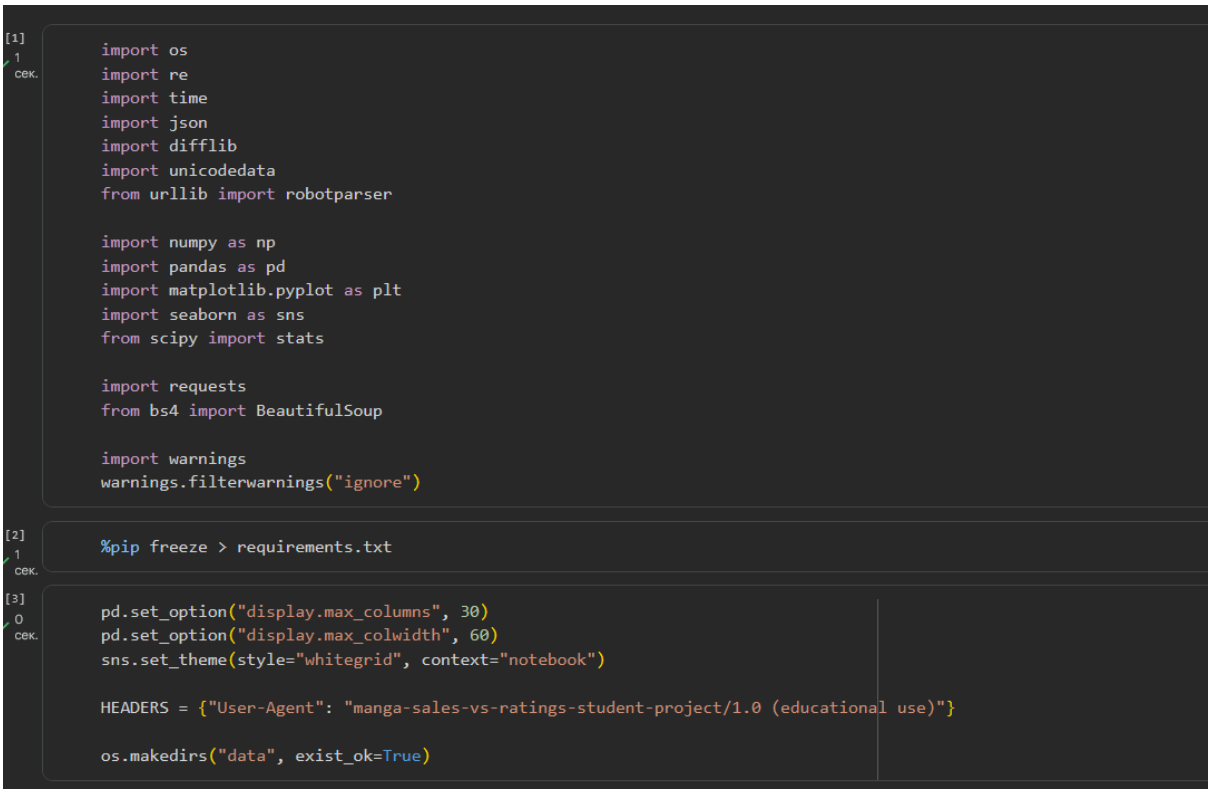
21.10.2026

Almaty

Setup - Libraries and Configuration

In this project I combined two sources: the **Jikan API** (JSON) and a **static Wikipedia page**. I used **requests** and **BeautifulSoup** for collecting the data, **pandas** and **numpy** for processing it, **scipy** for the statistical tests (Pearson, Spearman, Kruskal-Wallis), **difflib** and **unicodedata** for matching titles between the two sources, and **matplotlib** with **seaborn** for the plots. The package versions were saved to *requirements.txt* with *pip freeze*, so the environment can be reproduced.

I also set the pandas display options and a custom User-Agent header, which tells the servers who is sending the requests. All downloaded data is cached in the data folder, so the API does not have to be requested again every time the notebook is run.



```
[1]
1 сек.
import os
import re
import time
import json
import difflib
import unicodedata
from urllib import robotparser

import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns
from scipy import stats

import requests
from bs4 import BeautifulSoup

import warnings
warnings.filterwarnings("ignore")
```

```
[2]
1 сек.
%pip freeze > requirements.txt
```

```
[3]
0 сек.
pd.set_option("display.max_columns", 30)
pd.set_option("display.max_colwidth", 60)
sns.set_theme(style="whitegrid", context="notebook")

HEADERS = {"User-Agent": "manga-sales-vs-ratings-student-project/1.0 (educational use)"}

os.makedirs("data", exist_ok=True)
```

Task 1 - Project Topic, Data Sources and Hypotheses

The area I decided to use is **manga** - comic books from Japan. The goal of the project is to find out whether the commercial success of a manga series (how many copies it sold) correlates with how well it is received by readers, and whether the publisher and the genre/demographic of a series play a significant role in both.

API

I used the **Jikan API**, an open unofficial REST API for MyAnimeList (MAL): https://docs.api.jikan.moe/

Important: there was a global outage of the API when I started working on this project, so I had to use a backup version with slightly different functionality, called **Jikan Edge** (https://github.com/LucasHenriqueDiniz/jikan-edge). Two endpoints were used:

- *GET /v4/top/manga* - a paginated list of manga sorted by popularity, so that the sample is not biased towards only the highest-scored titles. Because of the outage, I replaced it with *v1/manga* sorted by the number of MAL members (the same idea) from Jikan Edge.
- *GET /v4/manga?q=<title>* - the search endpoint, used to find MAL records for series from the Wikipedia table that are not in the popularity list. It was replaced with *v1/manga?q=<title>*.

Jikan has a rate limit (3 requests per second and 60 per minute), so every request is followed by a pause of about one second.

Static web page

For the static page I used the Wikipedia article about the best-selling manga series, since it contains several structured tables with the sales numbers, publisher, demographic and number of volumes of every series:

https://en.wikipedia.org/wiki/List_of_best-selling_manga

Hypotheses

- **H1:** manga sales are positively correlated with the critical perception of the manga (MAL score, number of voters, members and favorites).
- **H2:** the publisher and the demographic/genre of a manga explain a significant share of the variation in sales and popularity.

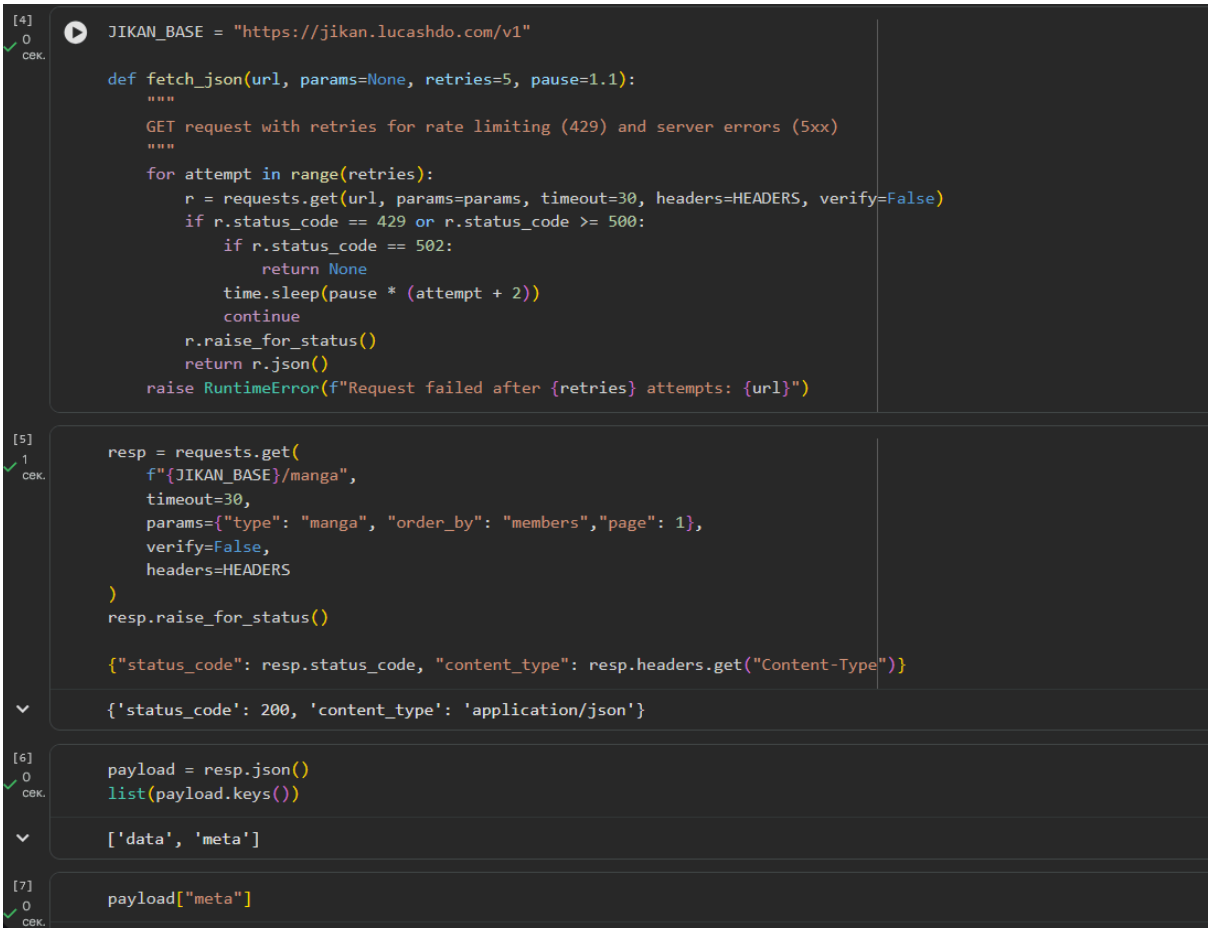
A note on terminology: the MAL score is an average of user ratings, not of professional critics, so I treat it as a proxy for the perception by the readers, not for any professional opinion.

Task 2 - API Data Collection

Requesting data from the selected API

First, I wrote a small helper, *fetch_json*, that sends a request, retries when the API answers with *429 Too Many Requests* or a server error, and returns the JSON. Then I requested the first page of the popularity list, checked the response status and examined the returned JSON. The status code is 200 and the content type is JSON, so the request was successful. I used *verify=False* to skip the SSL certificate check; this is acceptable for an educational task, but not for a production environment.

The response has two top-level keys - meta and data. The data key is an array of manga records and it is the main target of the analysis, while pagination in meta tells me how many records come per page (50) and whether there is a next page. A single record has fields like title, id, synopsis, user score, number of members, volumes and chapters. Since Berserk is an ongoing manga, it has None for the number of volumes and chapters.



```
[4]
0 сек.
JIKAN_BASE = "https://jikan.lucashdo.com/v1"

def fetch_json(url, params=None, retries=5, pause=1.1):
    """
    GET request with retries for rate limiting (429) and server errors (5xx)
    """
    for attempt in range(retries):
        r = requests.get(url, params=params, timeout=30, headers=HEADERS, verify=False)
        if r.status_code == 429 or r.status_code >= 500:
            if r.status_code == 502:
                return None
            time.sleep(pause * (attempt + 2))
            continue
        r.raise_for_status()
        return r.json()
    raise RuntimeError(f"Request failed after {retries} attempts: {url}")

[5]
1 сек.
resp = requests.get(
    f"{JIKAN_BASE}/manga",
    timeout=30,
    params={"type": "manga", "order_by": "members","page": 1},
    verify=False,
    headers=HEADERS
)
resp.raise_for_status()

{"status_code": resp.status_code, "content_type": resp.headers.get("Content-Type")}

{'status_code': 200, 'content_type': 'application/json'}

[6]
0 сек.
payload = resp.json()
list(payload.keys())

['data', 'meta']

[7]
0 сек.
payload["meta"]
```

Collecting the dataset from API

The list endpoint does not return much information, so I started by fetching the identifiers of manga titles and then requested the details of each of them separately.

I fetched 1000 titles (20 paginated requests, 50 results each) and saved the raw records as a JSON file with *json.dump*, so that they can be loaded again with *json.load*. If the

cache file exists, the data is loaded from it and not requested again. After that I requested the detailed record of every one of the 1000 titles, with a 1.1 second pause between requests. The detailed dataset has **995 records**: the helper skips a request when the server answers with a 502 error, so 5 titles could not be downloaded.



```
[9]
, 55
сек.

NUM_PAGES = 20
TOP_PARAMS = {"type": "manga", "order_by": "members"}
TOP_CACHE = "data/jikan_top_manga_raw.json"

IS_TOP_CACHED = os.path.exists(TOP_CACHE)

if IS_TOP_CACHED:
    with open(TOP_CACHE, "r", encoding="utf-8") as f:
        top_raw = json.load(f)
else:
    top_raw = []
    for page in range(1, NUM_PAGES + 1):
        payload = fetch_json(f"{JIKAN_BASE}/manga", params={**TOP_PARAMS, "page": page})
        top_raw.extend(payload["data"])
        if page % 10 == 0:
            print(f"Page {page} completed, {len(top_raw)} records so far")
        if not payload["meta"]["pagination"]["hasNextPage"]:
            break
        time.sleep(1.1)

    with open(TOP_CACHE, "w", encoding="utf-8") as f:
        json.dump(top_raw, f, ensure_ascii=False)

{"loaded_from_cache": IS_TOP_CACHED, "records": len(top_raw)}

Page 10 completed, 500 records so far
Page 20 completed, 1000 records so far
{'loaded_from_cache': False, 'records': 1000}
```

```
DETAILED_CACHE = "data/jikan_detailed_manga_raw.json"

IS_CACHED = os.path.exists(DETAILED_CACHE)

if IS_CACHED:
    with open(DETAILED_CACHE, "r", encoding="utf-8") as f:
        raw = json.load(f)
else:
    raw = []
    id_list = [manga["malId"] for manga in top_raw]
    print("Total manga records:", len(id_list))

    for mal_id in id_list:
        payload = fetch_json(f"{JIKAN_BASE}/manga/{mal_id}")
        if payload is None:
            print("Skipping manga with id:", mal_id)
            continue
        raw.append(payload["data"])
        if len(raw) % 10 == 0:
            print(f"{len(raw)} detailed manga records so far")
        time.sleep(1.1)

    with open(DETAILED_CACHE, "w", encoding="utf-8") as f:
        json.dump(raw, f, ensure_ascii=False)

{"loaded_from_cache": IS_CACHED, "records": len(raw)}
```

```
{'loaded_from_cache': True, 'records': 995}
```

Task 3 - JSON Processing

Flattening the JSON dataset

I started by flattening the nested records with *pd.json_normalize*. Nested dictionaries (like published) become dotted columns, while lists of dictionaries stay as they are. The result has 995 rows and 34 columns, and many of them are not needed for this project (image URLs, synopsis). I kept only the fields related to the sales/reception analysis: identifiers, titles, type, volumes and chapters, the reception metrics (score, members, rank), the publishing date and the lists of genres, authors, magazines and demographics.

There is also an important difference between the Jikan API and the Jikan Edge API: all columns in the Edge API have camelCase names. To keep everything more pythonic, I renamed them all to snake_case.

Extracting values from the nested lists

The lists of dictionaries need to be converted into usable columns, so I wrote small helpers: *names()* takes all the name values from a list, *first_name()* takes only the first one (for example, the first magazine is treated as the main serialization), *all_titles_of()* collects all

known titles of a series, and *norm_demo()* maps the demographic labels to one standard spelling, because the sources differ ("Shounen" in MAL, "Shōnen" in Wikipedia).

Now every record has simple columns (demographic, magazine, author, start_year, primary_genre) and two list columns (genre_list, theme_list) which keep all genres and themes of a manga. The list columns make *explode* useful later: it creates one row per manga-genre pair, which can be used to analyze the distribution of genres across manga titles.

```python
def names(lst):
    return [d["name"] for d in lst] if isinstance(lst, list) else []

def first_name(lst):
    return lst[0]["name"] if isinstance(lst, list) and len(lst) > 0 else np.nan

def norm_demo(s):
    """
    Standardise demographic labels from both sources (Shounen/Shōnen -> Shonen, etc.)
    """
    if pd.isna(s):
        return np.nan
    t = s.lower().replace("ō", "o").replace("ū", "u").replace("ou", "o")
    for key in ["shonen", "seinen", "shojo", "josei"]:
        if key in t:
            return key.capitalize()
    if "kids" in t or "kodomo" in t:
        return "Kodomo"
    return "Other"

def all_titles_of(rec):
    """Return list of all title - official English title and alt titles"""
    out = [rec.get("title"), rec.get("titleEnglish")]
    out += rec.get("titleSynonyms", []) or []
    return [t for t in dict.fromkeys(out) if t]

def to_frame(records):
    df = pd.json_normalize(records)
    keep = ["malId", "title", "titleEnglish", "type", "chapters", "volumes", "status", "publishing",
            "score", "scoredBy", "rank", "popularity", "members", "favorites", "published.from",
            "authors", "serializations", "genres", "themes", "demographics"]
    df = df[[c for c in keep if c in df.columns]].rename(
        columns={
            "published.from": "published_from",
            "malId": "mal_id",
            "title": "title",
            "titleEnglish": "title_english",
            "type": "type",
            "chapters": "chapters",
            "volumes": "volumes",
            "publishing": "publishing",
            "score": "score",
            "scoredBy": "scored_by",
            "rank": "rank",
            "popularity": "popularity",
            "members": "members",
            "favorites": "favorites",
            "authors": "authors",
            "serializations": "serializations",
            "genres": "genres",
            "themes": "themes",
            "demographics": "demographics"
        }
    )

    df["all_titles"] = [all_titles_of(r) for r in records]
    df["genre_list"] = df["genres"].apply(names)
    df["theme_list"] = df["themes"].apply(names)
    df["primary_genre"] = df["genre_list"].apply(lambda g: g[0] if g else np.nan)
    df["demographic"] = df["demographics"].apply(first_name).map(norm_demo)
    df["magazine"] = df["serializations"].apply(first_name)
    df["author"] = df["authors"].apply(first_name)
    df["start_year"] = pd.to_datetime(df["published_from"], errors="coerce", utc=True).dt.year
```

	mal_id	title	title_english	type	chapters	volumes	status	publishing	score	scored_by	rank	popularity	members	favorites	all_titles	genre_list	theme_list	primary_genre	demographic	magazine	auth
0	2	Berserk	Berserk	Manga	NaN	NaN	Publishing	True	9.46	406793	1.0	1	813179	139874	[Berserk, Berserk: The Prototype]	[Action, Adventure, Award Winning, Drama, Fantasy, Horror]	[Gore, Military, Psychological]	Action	Seinen	Young Animal	Miu Kentar
1	116778	Chainsaw Man	Chainsaw Man	Manga	232.0	24.0	Finished	False	8.61	458399	108.0	2	750865	92580	[Chainsaw Man]	[Action, Award Winning, Fantasy]	[Gore, Urban Fantasy]	Action	Shonen	Shounen Jump+	Fujimo Tats
2	13	One Piece	One Piece	Manga	NaN	NaN	Publishing	True	9.20	427636	4.0	3	710237	127219	[One Piece]	[Action, Adventure, Fantasy]	[]	Action	Shonen	Shounen Jump (Weekly)	Oc Eiich
3	23390	Shingeki no Kyojin	Attack on Titan	Manga	141.0	34.0	Finished	False	8.56	434694	135.0	4	699821	70072	[Shingeki no Kyojin, Attack on Titan]	[Action, Award Winning, Drama, Suspense]	[Gore, Military, Survival]	Action	Shonen	Bessatsu Shounen Magazine	Isayam Haji
4	33327	Tokyo Ghoul	Tokyo Ghoul	Manga	144.0	14.0	Finished	False	8.56	316840	136.0	6	580422	52253	[Tokyo Ghoul, Toukyou Kushu]	[Action, Fantasy, Horror]	[Gore, Psychological, Urban Fantasy]	Action	Seinen	Young Jump	Ishi

```
df_tmp[["title", "genre_list"]].explode("genre_list").head(8)
```

	title	genre_list
0	Berserk	Action
0	Berserk	Adventure
0	Berserk	Award Winning
0	Berserk	Drama
0	Berserk	Fantasy
0	Berserk	Horror
1	Chainsaw Man	Action
1	Chainsaw Man	Award Winning

Missing values, data types and deduplication

Empty strings were turned into NaN, identifiers and counters were converted to nullable integers (Int64), and the categorical columns (type, status, demographic) to category. The table below shows the missing values of the popularity list (995 rows).

Column	Missing	Comment
chapters	236	ongoing series or no data
volumes	231	ongoing series or no data
demographic	214	not filled in on MAL for some series
title_english	131	no official English title
start_year	42	no publishing date
rank	33	not ranked
magazine	17	no serialization listed
all other columns	0	

The score had no missing values in this list, but it does have 20 missing values in the final dataset (after the extra series were added in Task 5): a missing score means the series

was not rated by enough users, and these series have to be left out of any correlation analysis. A missing demographic is simply not filled in for some series on MAL.

Since the pagination of a live list can shift between requests, the same series may appear twice, so I deduplicated by the MAL identifier. There were 0 duplicates, and the final shape is (995, 22).

```
df_clean = clean_api(df_tmp)
df_clean.isna().sum()
```

	0
mal_id	0
title	0
title_english	131
type	0
chapters	236
volumes	231
status	0
publishing	0
score	0
scored_by	0
rank	33
popularity	0
members	0
favorites	0
all_titles	0
genre_list	0
theme_list	0
primary_genre	0
demographic	214
magazine	17
author	0
start_year	42

```
print("Duplicate rows (by mal_id):", df_clean.duplicated(subset="mal_id").sum())
df_top = df_clean.drop_duplicates(subset="mal_id").reset_index(drop=True)
df_top.shape
```

```
Duplicate rows (by mal_id): 0
(995, 22)
```

```
df_top.head()
```

	mal_id	title	title_english	type	chapters	volumes	status	publishing	score	scored_by	rank	popularity	members	favorites	all_titles	genre_list	theme_list	primary_genre	demographic	magazine	author	st
0	2	Berserk	Berserk	Manga	<NA>	<NA>	Publishing	True	9.46	406793	1	1	813179	139874	[Berserk, Berserk: The Prototype]	[Action, Adventure, Award Winning, Drama, Fantasy, Horror]	[Gore, Military, Psychological]	Action	Seinen	Young Animal	Miura, Kentarou	
1	116778	Chainsaw Man	Chainsaw Man	Manga	232	24	Finished	False	8.61	458399	108	2	750865	92580	[Chainsaw Man]	[Action, Award Winning, Fantasy]	[Gore, Urban Fantasy]	Action	Shonen	Shounen Jump+	Fujimoto, Tatsuki	
2	13	One Piece	One Piece	Manga	<NA>	<NA>	Publishing	True	9.20	427636	4	3	710237	127219	[One Piece]	[Action, Adventure, Fantasy]	[]	Action	Shonen	Shounen Jump (Weekly)	Oda, Eiichiro	
3	23390	Shingeki no Kyojin	Attack on Titan	Manga	141	34	Finished	False	8.56	434694	135	4	699821	70072	[Shingeki no Kyojin, Attack on Titan]	[Action, Award Winning, Drama, Suspense]	[Gore, Military, Survival]	Action	Shonen	Bessatsu Shounen Magazine	Isayama, Hajime	
4	33327	Tokyo Ghoul	Tokyo Ghoul	Manga	144	14	Finished	False	8.56	316840	136	6	580422	52253	[Tokyo Ghoul, Toukyou Kushu]	[Action, Fantasy, Horror]	[Gore, Psychological, Urban Fantasy]	Action	Seinen	Young Jump	Ishida, Sui	

Task 4 - Web Scraping

Downloading the page and finding the tables

I requested the Wikipedia page with the requests library (status code 200) and used BeautifulSoup to find all the tables of the wikitable class. There are 4 tables on the page, and all of them have the same columns: manga series, authors, publisher, demographic, number of collected volumes, serialization period and approximate sales.

Writing a table parser

Wikipedia tables are not trivial to parse: cells contain footnote markers ([a]), line breaks between several values, and rowspan cells (for example, a publisher name written once for several consecutive series). A simple "one td = one column" loop would shift the columns in such rows. Therefore I wrote a parser, *parse_table()*, that:

- removes footnotes (sup) and style tags from the cell text,
- replaces line breaks with a separator,
- remembers cells with rowspan/colspan and repeats their value in the following rows.

The column names are not exactly the same in every table, so *map_columns()* maps every header to a standard name by keywords (series, author, publisher, demographic, volumes_raw, years_raw, sales_raw). Every table also remembers the section heading it is under, which is saved as wiki_section. The raw result has 201 rows and 8 columns, one row per series.

```
def clean_text(text):
    text = re.sub(r"\[[^\]]*\]", "", text) # removing foot notes
    text = re.sub(r"†", "", text) # dunno what these mean
    text = re.sub(r"‡", "", text)
    return re.sub(r"\s+", " ", text).strip() # removing spaces


def cell_text(cell):
    for tag in cell.find_all(["sup", "style"]):
        tag.decompose()
    for br in cell.find_all("br"):
        br.replace_with(" | ")
    return clean_text(cell.get_text(" ", strip=True))


def parse_table(table):
    """
    Returns the table as a list of rows (list of strings), expanding rowspan/colspan
    """
    rows, spans = [], {}
    for tr in table.find_all("tr"):
        cells = tr.find_all(["th", "td"], recursive=False)
        row, col, ci = [], 0, 0
        while ci < len(cells) or col in spans:
            if col in spans:
                text, left = spans[col]
                row.append(text)
                if left > 1:
                    spans[col] = (text, left - 1)
                else:
                    del spans[col]
                col += 1
                continue
            cell = cells[ci]
            ci += 1
            text = cell_text(cell)
            rs = int(re.sub(r"\D", "", cell.get("rowspan", "1")) or 1)
            cs = int(re.sub(r"\D", "", cell.get("colspan", "1")) or 1)
            for _ in range(cs):
                row.append(text)
                if rs > 1:
                    spans[col] = (text, rs - 1)
                col += 1
        if row:
            rows.append(row)
    return rows
```

```python
def map_columns(headers):
    """Map the index of every header to a standard column name by keywords."""
    rules = [("sales_raw", ["sales", "copies"]), ("author", ["author", "creator"]),
             ("publisher", ["publisher"]), ("demographic", ["demograph"]),
             ("volumes_raw", ["volume"]), ("years_raw", ["serializ", "year", "period"]),
             ("series", ["series", "title", "manga"])]
    mapping, used = {}, set()
    for i, h in enumerate(headers):
        hl = h.lower()
        for key, words in rules:
            if key not in used and any(w in hl for w in words):
                mapping[i] = key
                used.add(key)
                break
    return mapping

def section_of(table):
    h = table.find_previous(["h2", "h3"])
    return clean_text(h.get_text(" ", strip=True)) if h else np.nan

records = []
for t in tables:
    parsed = parse_table(t)
    if len(parsed) < 2:
        continue
    headers = parsed[0]
    cmap = map_columns(headers)
    if not {"series", "sales_raw"} <= set(cmap.values()):
        continue
    section = section_of(t)
    for r in parsed[1:]:
        if len(r) != len(headers):
            continue
        rec = {cmap[i]: r[i] for i in cmap}
        rec["wiki_section"] = section
        records.append(rec)

df_wiki_raw = pd.DataFrame(records)
df_wiki_raw.shape
```

```python
df_wiki_raw.head(10)
```

	series	author	publisher	demographic	volumes_raw	years_raw	sales_raw	wiki_section
0	One Piece	Eiichiro Oda	Shueisha	Shōnen	115	1997–present	600 million	At least 100 million copies
1	Doraemon	Fujiko F. Fujio	Shogakukan	Children	45	1969–1996	300 million	At least 100 million copies
2	Golgo 13	Takao Saito , Saito Production	Shogakukan	Seinen	221	1968–present	300 million	At least 100 million copies
3	Case Closed / Detective Conan	Gosho Aoyama	Shogakukan	Shōnen	108	1994–present	270 million	At least 100 million copies
4	Dragon Ball	Akira Toriyama	Shueisha	Shōnen	42	1984–1995	260 million	At least 100 million copies
5	Naruto	Masashi Kishimoto	Shueisha	Shōnen	72	1999–2014	250 million	At least 100 million copies
6	Demon Slayer: Kimetsu no Yaiba	Koyoharu Gotouge	Shueisha	Shōnen	23	2016–2020	220 million	At least 100 million copies
7	Slam Dunk	Takehiko Inoue	Shueisha	Shōnen	31	1990–1996	185 million	At least 100 million copies
8	KochiKame: Tokyo Beat Cops	Osamu Akimoto	Shueisha	Shōnen	201	1976–2016	157.2 million	At least 100 million copies
9	Jujutsu Kaisen	Gege Akutami	Shueisha	Shōnen	30	2018–2024	150 million	At least 100 million copies

Cleaning the scraped data

The raw table contains only text, so the numeric fields had to be parsed:

- **sales_m** - approximate sales converted to millions of copies (handles "million" and "billion" words and thousands separators),
- **volumes_wiki** - the first number in the volumes column,
- **start_year_wiki** - the first four-digit year of the serialization period,

- **publisher** - only the first publisher is kept, the parentheses are removed and several spellings are mapped to one name,
- **demographic** - standardized with the same norm_demo function as in the API data.

There are no rows without parsed sales and no duplicate series, so all 201 series were kept and the final shape of the scraped dataset is (201, 11) before matching.

```python
def parse_sales(s):
    """'516.6 million' -> 516.6, '1.2 billion' -> 1200.0 (millions of copies)."""
    if pd.isna(s):
        return np.nan
    m = re.search(r"(\d[\d,]*\.?\d*)", s)
    if not m:
        return np.nan
    val = float(m.group(1).replace(",", ""))
    low = s.lower()
    if "billion" in low:
        return val * 1000
    if "million" in low:
        return val
    return val / 1e6 if val > 10_000 else val

PUB_MAP = {
    "square enix": "Square Enix",
    "enix": "Square Enix",
    "kadokawa": "Kadokawa",
    "shueisha": "Shueisha",
    "kodansha": "Kodansha",
    "shogakukan": "Shogakukan",
    "akita": "Akita Shoten",
    "hakusensha": "Hakusensha",
    "futabasha": "Futabasha"
}

def primary_publisher(s):
    if pd.isna(s):
        return np.nan
    s = re.sub(r"\(.*?\)", "", s)
    first = re.split(r"\s*(?:\||,|/|;)\s*", s)[0].strip()
    for key, name in PUB_MAP.items():
        if key in first.lower():
            return name
    return first or np.nan
```

```
df_wiki = df_wiki_raw.copy()
for col in ["series", "author", "publisher", "demographic", "volumes_raw", "years_raw", "sales_raw"]:
    if col not in df_wiki.columns:
        df_wiki[col] = np.nan
df_wiki = df_wiki.replace(r"^\s*$", np.nan, regex=True)

df_wiki["sales_m"] = df_wiki["sales_raw"].map(parse_sales)
df_wiki["volumes_wiki"] = pd.to_numeric(df_wiki["volumes_raw"].str.extract(r"(\d+)")[0], errors="coerce")
df_wiki["start_year_wiki"] = pd.to_numeric(df_wiki["years_raw"].str.extract(r"(\d{4})")[0], errors="coerce")
df_wiki["publisher"] = df_wiki["publisher"].map(primary_publisher)
df_wiki["demographic"] = df_wiki["demographic"].map(norm_demo)
df_wiki["series"] = df_wiki["series"].str.replace(r"[*†‡]+$", "", regex=True).str.strip()

df_wiki.isna().sum()
```

	0
series	0
author	0
publisher	0
demographic	0
volumes_raw	0
years_raw	0
sales_raw	0
wiki_section	0
sales_m	0
volumes_wiki	0
start_year_wiki	0

dtype: int64

```
print("Rows without parsed sales:", df_wiki["sales_m"].isna().sum())
print("Duplicate series:", df_wiki.duplicated(subset="series").sum())

df_wiki = df_wiki \
    .dropna(subset=["series", "sales_m"]) \
    .drop_duplicates(subset="series") \
    .reset_index(drop=True)
df_wiki.shape
```

```
Rows without parsed sales: 0
Duplicate series: 0
(201, 11)
```

```
df_wiki[["series", "author", "publisher", "demographic", "volumes_wiki", "start_year_wiki", "sales_m", "wiki_section"]].head(10)
```

	series	author	publisher	demographic	volumes_wiki	start_year_wiki	sales_m	wiki_section
0	One Piece	Eiichiro Oda	Shueisha	Shonen	115	1997	600.0	At least 100 million copies
1	Doraemon	Fujiko F. Fujio	Shogakukan	Other	45	1969	300.0	At least 100 million copies
2	Golgo 13	Takao Saito , Saito Production	Shogakukan	Seinen	221	1968	300.0	At least 100 million copies
3	Case Closed / Detective Conan	Gosho Aoyama	Shogakukan	Shonen	108	1994	270.0	At least 100 million copies
4	Dragon Ball	Akira Toriyama	Shueisha	Shonen	42	1984	260.0	At least 100 million copies
5	Naruto	Masashi Kishimoto	Shueisha	Shonen	72	1999	250.0	At least 100 million copies
6	Demon Slayer: Kimetsu no Yaiba	Koyoharu Gotouge	Shueisha	Shonen	23	2016	220.0	At least 100 million copies
7	Slam Dunk	Takehiko Inoue	Shueisha	Shonen	31	1990	185.0	At least 100 million copies
8	KochiKame: Tokyo Beat Cops	Osamu Akimoto	Shueisha	Shonen	201	1976	157.2	At least 100 million copies
9	Jujutsu Kaisen	Gege Akutami	Shueisha	Shonen	30	2018	150.0	At least 100 million copies

Task 5 - Merging the Two Datasets

Matching series between Wikipedia and MyAnimeList

The two sources have no shared identifier, so the only way to join them is through the titles. The same series is written differently in the two sources ("Case Closed" / "Detective Conan" / "Meitantei Conan", special characters, subtitles), therefore the matching was done in steps:

- **Normalization:** titles are lowercased, accents and punctuation are removed.
- **Exact match** against all known MAL titles of the popularity list (default, English, and synonyms).
- **Search fallback:** for the series that are still unmatched I asked the Jikan search endpoint and took the candidate with the highest string similarity (difflib), accepting it only if the similarity is at least 0.85. The search responses are cached in a JSON file as well.

Match method	Series
exact	96
search (similarity >= 0.85)	68
unmatched	37
total	201

There are only 96 exact matches after title normalization, so more than half of the records from the Wikipedia page were not easily mapped to the MAL dataset. The unmatched list included well-known series like Detective Conan, Rurouni Kenshin and Index, which was strange, so the cutoff was too strict.

Less strict matching

For the remaining 37 series I used a more relaxed approach: the title is normalized again (removing non-ASCII characters and punctuation, as in a manual search), only the first 3 words are used as the search query, the full Wikipedia name is compared with every title of every candidate, and the similarity cutoff is lowered to 0.3. A relaxed cutoff produces false matches, so I checked the result manually. Several matches were wrong or doubtful (for example, Keiji was matched to Keijo, and short-word titles like Shoot! and Gaki Deka were matched incorrectly), so I manually set the MAL id to missing for 8 records: Shoot!, Crest of the Royal Family, Gaki Deka, Kaiji, Boys Be..., Oh My Goddess!, Himitsu Series and Don't Call It Mystery. Some series (for example, KochiKame, Dobutsu no Oisha, Shonen Shojo Nippon) were not found at all.

```python
def norm_title(s):
    s = unicodedata.normalize("NFKD", str(s))
    s = "".join(ch for ch in s if not unicodedata.combining(ch)).lower().replace("&", " and ")
    return re.sub(r"[^a-z0-9]+", " ", s).strip()

def clean_query(s):
    return re.sub(r"\s+", " ", re.sub(r"\(.*?\)|\[.*?\]", "", s)).strip()

# normalized title -> mal_id; the most popular record wins when two records share a title
title_lookup = {}
for _, row in df_top.sort_values("members", ascending=False).iterrows():
    for t in row["all_titles"]:
        title_lookup.setdefault(norm_title(t), row["mal_id"])

def exact_match(series):
    return title_lookup.get(norm_title(series)) or title_lookup.get(norm_title(clean_query(series)))

df_wiki["mal_id"] = df_wiki["series"].map(exact_match).astype("Int64")
exact_mask = df_wiki["mal_id"].notna()

print("Series Total:", len(df_wiki))
print("Exact Matches:", int(exact_mask.sum()))
```

```
Series Total: 201
Exact Matches: 96
```

```python
found, ratios = {}, {}

for s in unmatched:
    q = MANUAL_QUERIES.get(s, clean_query(s))
    best, ratio = best_candidate(q, search_manga(q))
    ratios[s] = round(ratio, 2)
    if best is not None:
        found[s] = best

with open(SEARCH_CACHE, "w", encoding="utf-8") as f:
    json.dump(search_cache, f, ensure_ascii=False)

{"searched": len(unmatched), "found_by_search": len(found)}
```

```
{'searched': 105, 'found_by_search': 68}
```

```python
df_wiki["mal_id"] = df_wiki["mal_id"].fillna(df_wiki["series"].map({s: b["malId"] for s, b in found.items()})).astype("Int64")
df_wiki["match_method"] = np.where(exact_mask, "exact", np.where(df_wiki["series"].isin(found), "search", "unmatched"))

df_wiki["match_method"].value_counts()
```

match_method	count
exact	96
search	68
unmatched	37

dtype: int64

```python
RELAXED_CUTOFF = 0.3
RELAXED_WORDS = 3

def ascii_fold(s):
    """
    Removes non-ascii characters, apostrophes, punctuation
    to emulate manual search
    """
    s = unicodedata.normalize("NFKD", str(s))
    s = "".join(ch for ch in s if not unicodedata.combining(ch))
    s = re.sub(r"['’]", "", s)                # JoJo's -> JoJos
    s = re.sub(r"[^A-Za-z0-9 ]+", " ", s)     # &, !, ., /, - , ... -> space
    return re.sub(r"\s+", " ", s).strip()

def first_words(s, n=RELAXED_WORDS):
    no_the = re.sub("The ", "", s)
    return " ".join(ascii_fold(no_the).split()[:n])

def cand_titles(c):
    """
    Get all titles available in a candidate record
    """
    out = [c.get("title"), c.get("titleEnglish"), c.get("title_english")]
    out += c.get("titleSynonyms") or c.get("title_synonyms") or []
    return [t for t in out if t]

def cand_id(c):
    return c.get("malId") or c.get("mal_id")

def search_relaxed(query):
    if query not in search_cache:
        payload = fetch_json(
            f"{JIKAN_BASE}/manga",
            params={"q": query, "type": "manga", "order_by": "members"}
        )
        search_cache[query] = payload["data"][:10]
        time.sleep(1.1)
    return search_cache[query]

def best_relaxed(series, candidates):
    """
    Compares the FULL cleaned Wikipedia name with every title of every candidate
    """

    targets = [norm_title(ascii_fold(p)) for p in re.split(r"\s*/\s*", series) if p.strip()]
    best, best_title, best_ratio = None, None, 0.0
    for c in candidates:
        if c.get("type") in ("Novel", "Light Novel"):
            continue
        for t in cand_titles(c):
            nt = norm_title(ascii_fold(t))
            ratio = max(difflib.SequenceMatcher(None, tg, nt).ratio() for tg in targets)
            if ratio > best_ratio:
                best, best_title, best_ratio = c, t, ratio
    return best, best_title, best_ratio
```

```
relaxed_report = pd.DataFrame(relaxed_rows).sort_values("similarity", ascending=False)
relaxed_report
```

	series	query	fetched_title	mal_id	similarity	accepted
14	Kyō Kara Ore Wa!!	Kyo Kara Ore	Kyou kara Ore wa!!	6812.0	0.97	True
25	Tasogare Ryūseigun	Tasogare Ryuseigun	Tasogare Ryuuseigun	43157.0	0.97	True
20	Ushio & Tora	Ushio Tora	Ushio to Tora	7747.0	0.87	True
34	Keiji	Keiji	Keijo!!!!!!!	56505.0	0.80	True
15	Hell Teacher: Jigoku Sensei Nube	Hell Teacher Jigoku	Jigoku Sensei Nube S	115079.0	0.71	True
5	Rurouni Kenshin	Rurouni Kenshin	Rurouni Kenshin: Tokuhitsuban	37303.0	0.70	True
0	Case Closed / Detective Conan	Case Closed	Meitantei Conan	1061.0	0.67	True
7	Dragon Quest: The Adventure of Dai	Dragon Quest Adventure	Dragon Quest Treasures: Another Adventure - Fadora no Ta...	177190.0	0.66	True
19	Kaiji	Kaiji	Kaiji Gaiden	94100.0	0.59	True
9	Glass Mask	Glass Mask	Glass no Kamen	1469.0	0.58	True
12	Kosaku Shima	Kosaku Shima	Kachou Shima Kousaku	17204.0	0.56	True
3	Astro Boy	Astro Boy	Astro Boy: Tetsuwan Atom	3010.0	0.56	True
4	The Kindaichi Case Files	Kindaichi Case Files	Kindaichi Shounen no Jikenbo: File Series	393.0	0.56	True
8	Fisherman Sanpei	Fisherman Sanpei	Tsurikichi Sanpei	10054.0	0.55	True
33	Haruhi Suzumiya	Haruhi Suzumiya	Haruhi Comic Anthology	84435.0	0.54	True
30	Dragon Quest: The Mark of Erdick	Dragon Quest Mark	Dragon Quest Retsuden: Roto no Monshou	5663.0	0.53	True
17	A Certain Magical Index	A Certain Magical	Toaru Majutsu no Index	7775.0	0.49	True
28	Don't Call It Mystery	Dont Call It	Hands Off!: Don't Call Us Angels	300.0	0.49	True
26	Himitsu Series	Himitsu Series	Arcana - Tousou / Himitsu Kessha	24480.0	0.48	True
27	Ushijima the Loan Shark	Ushijima the Loan	Yamikin Ushijima-kun	8593.0	0.47	True

Building the final dataset and merging

The records found through the search (97 identifiers, of which 94 were downloaded; 3 were skipped because of server errors) were added to the popularity list, so the final API dataset consists of the 995 most popular manga and the best-sellers that were not in it: **1082 rows and 22 columns** after removing duplicates. After that I used *merge* with *how="left"* and *indicator=True*, which shows how many Wikipedia rows have a MAL partner.

_merge value	Rows
both (matched)	183
left_only (Wikipedia only)	18
right_only	0

The 18 unmatched Wikipedia rows are the 16 series without a MAL id and, presumably, 2 series whose detailed records could not be downloaded. Two different Wikipedia rows could in theory point to the same MAL record, so I kept only the row with the higher sales for each MAL identifier. The final analysis dataframe df_m has 183 matched series (91% of the 201 best-sellers) and 40 columns.

I also created several new columns in this step:

- **demographic** - one column (the Wikipedia value, and MAL as a fallback),
- **publisher_grp** - the small publishers (fewer than 3 series) are grouped into "Other", otherwise the group statistics for publishers with a single series would be meaningless,

- **log_sales** and **log_members** - log10 versions of the strongly right-skewed variables,
- **sales_per_volume_m** - millions of copies per volume, which separates "a long series that sold a lot because it has 100+ volumes" from "a series that sells a lot per volume",
- **n_genres** - the number of genres of a series.

Finally, I saved three datasets to CSV: jikan_manga.csv (API), wiki_best_selling_manga.csv (web) and manga_merged.csv (merged).

```
EXTRA_CACHE = "data/jikan_extra_manga_raw.json"

IS_EXTRA_CACHED = os.path.exists(EXTRA_CACHE)

if IS_EXTRA_CACHED:
    with open(EXTRA_CACHE, "r", encoding="utf-8") as f:
        extra_raw = json.load(f)
else:
    extra_raw = []
    print("Total found manga records:", len(found_ids))

    for mal_id in found_ids:
        payload = fetch_json(f"{JIKAN_BASE}/manga/{mal_id}")
        if payload is None:
            print("Skipping manga with id:", mal_id)
            continue
        extra_raw.append(payload["data"])
        if len(extra_raw) % 10 == 0:
            print(f"{len(extra_raw)} detailed manga records so far")
        time.sleep(1.1)

    with open(EXTRA_CACHE, "w", encoding="utf-8") as f:
        json.dump(extra_raw, f, ensure_ascii=False)

{"loaded_from_cache": IS_EXTRA_CACHED, "records": len(extra_raw)}
```

```
Total found manga records: 97
Skipping manga with id: 3483
10 detailed manga records so far
Skipping manga with id: 21308
20 detailed manga records so far
30 detailed manga records so far
40 detailed manga records so far
50 detailed manga records so far
60 detailed manga records so far
70 detailed manga records so far
80 detailed manga records so far
Skipping manga with id: 94100
90 detailed manga records so far
{'loaded_from_cache': False, 'records': 94}
```

```
df_api = clean_api(to_frame(raw + extra_raw)).drop_duplicates(subset="mal_id").reset_index(drop=True)
df_api.shape
```

```
(1082, 22)
```

```
df_all = df_wiki.merge(df_api, on="mal_id", how="left", suffixes=("_wiki", "_mal"), indicator=True)
df_all["_merge"].value_counts()
```

```
df_all = df_wiki.merge(df_api, on="mal_id", how="left", suffixes=("_wiki", "_mal"), indicator=True)
df_all["_merge"].value_counts()
```

_merge	count
both	183
left_only	18
right_only	0

dtype: int64

Two different Wikipedia rows could in theory point to the same MAL record (for example a series and its sequel with a very similar name). I keep only the row with the higher sales for each MAL identifier, and then build the final analysis dataframe `df_m` with only the successfully matched series.

+ Код + Текст

```
df_m = (df_all[df_all["_merge"] == "both"]
        .sort_values("sales_m", ascending=False)
        .drop_duplicates(subset="mal_id")
        .drop(columns="_merge")
        .reset_index(drop=True))

df_m["demographic"] = df_m["demographic_wiki"].fillna(df_m["demographic_mal"].astype("object"))

# nullable ints -> float
for col in ["scored_by", "rank", "popularity", "members", "favorites", "volumes", "chapters", "volumes_wiki", "start_year_wiki"]:
    df_m[col] = df_m[col].astype(float)

df_m["publisher_grp"] = df_m["publisher"].where(df_m.groupby("publisher")["publisher"].transform("size") >= 3, "Other")
df_m["log_sales"] = np.log10(df_m["sales_m"])
df_m["log_members"] = np.log10(df_m["members"])
df_m["sales_per_volume_m"] = df_m["sales_m"] / df_m["volumes_wiki"].replace(0, np.nan)
df_m["n_genres"] = df_m["genre_list"].str.len()

df_m.shape
```

(183, 40)

Task 6 - Data Review

Structure

I defined a helper function that prints the shape, the data types and the columns with the most missing values for a DataFrame, and applied it to all three datasets.

	API (Jikan)	Web (Wikipedia)	Merged
Rows	1082	201	183
Columns	22	13	40
Rows with MAL score	1062	-	165
Distinct primary genres	20	-	9
Distinct magazines / publishers	230 magazines	-	15 publishers

Dataset	Columns with the most missing values
API (Jikan)	chapters 255, volumes 242, demographic 222, title_english 178, start_year 75, rank 34, magazine 24, score 20
Web (Wikipedia)	mal_id 16 (series without a MAL partner); all scraped columns are complete

The API dataset is large (about a thousand manga) and describes the readers' perception and the genres. The Wikipedia dataset is small (it only contains the best-sellers), but it is the only source of sales and publishers. The merged dataset has as many rows as

there are best-sellers that I managed to find on MAL, so all inferences about sales are based on this smaller sample of 183 series (165 of them have a score).

```
pd.DataFrame({
    "API (Jikan)":      [df_api.shape[0], df_api.shape[1], int(df_api["score"
    "Web (Wikipedia)":  [df_wiki.shape[0], df_wiki.shape[1], np.nan, np.nan,
    "Merged":           [df_m.shape[0], df_m.shape[1], int(df_m["score"].notn
}, index=["rows", "columns", "rows with MAL score", "distinct primary genres"
```

	API (Jikan)	Web (Wikipedia)	Merged
rows	1082	201.0	183
columns	22	13.0	40
rows with MAL score	1062	NaN	165
distinct primary genres	20	NaN	9
distinct magazines / publishers	230	NaN	15

Task 7 - Exploratory Data Analysis

Descriptive statistics

Variable	count	mean	std	min	median	max
sales_m (millions)	183	54.13	64.26	20	32	600
volumes_wiki	183	45.31	36.52	5	34	221
score	165	8.00	0.58	6.64	8.08	9.46
scored_by	165	56,215	90,466	107	16,310	458,399
members	183	104,338	158,419	31	28,429	813,179
favorites	183	8,166	18,429	0	992	139,874
sales_per_volume_m	183	1.49	1.40	0.21	1.09	10.00

Sales, members, favorites and the number of voters are strongly right-skewed (the mean is much bigger than the median, and the maximum is many times higher than the 75th percentile): a handful of mega-franchises are far ahead of the rest. The MAL score, on the other hand, is concentrated in a narrow range (mean 8.00, standard deviation 0.58), because the series that make it to a best-selling list are typically well rated. This is the reason why I use log transformations and rank-based (Spearman) correlations in the next steps.

Indexing

First, I selected the top 10 best-selling series with the most relevant columns using *loc* (label-based: rows by index and columns by name), and the first 5 rows and 3 columns with *iloc* (position-based). *idxmax* returned the index of the best-selling series, One Piece.

Series	Publisher	Demographic	Sales (M)	Vol.	Score	Members
One Piece	Shueisha	Shonen	600	115	9.20	710,237

Series	Publisher	Demographic	Sales (M)	Vol.	Score	Members
Doraemon	Shogakukan	Other	300	45	8.45	18,255
Golgo 13	Shogakukan	Seinen	300	221	7.69	7,647
Case Closed / Detective Conan	Shogakukan	Shonen	270	108	8.29	65,534
Dragon Ball	Shueisha	Shonen	260	42	8.46	186,671
Naruto	Shueisha	Shonen	250	72	8.09	449,527
Demon Slayer: Kimetsu no Yaiba	Shueisha	Shonen	220	23	8.09	508,330
Slam Dunk	Shueisha	Shonen	185	31	9.09	204,691
Jujutsu Kaisen	Shueisha	Shonen	150	30	8.17	525,383
Crayon Shin-chan	Futabasha	Seinen	148	65	7.97	6,874

The table shows that being at the top of the sales list does not always mean being popular on MAL: Doraemon, Golgo 13 and Crayon Shin-chan sold 148-300 million copies, but have only about 7,000-18,000 MAL members, while Naruto, Demon Slayer and Jujutsu Kaisen have 450,000-525,000 members. Most likely, this is because these older titles sold mainly in Japan, while MAL is an international online community.

```
top10 = df_m.sort_values("sales_m", ascending=False).reset_index(drop=True)
top10.loc[:9, ["series", "publisher", "demographic", "sales_m", "volumes_wiki", "score", "members"]]
```

	series	publisher	demographic	sales_m	volumes_wiki	score	members
0	One Piece	Shueisha	Shonen	600.0	115.0	9.20	710237.0
1	Doraemon	Shogakukan	Other	300.0	45.0	8.45	18255.0
2	Golgo 13	Shogakukan	Seinen	300.0	221.0	7.69	7647.0
3	Case Closed / Detective Conan	Shogakukan	Shonen	270.0	108.0	8.29	65534.0
4	Dragon Ball	Shueisha	Shonen	260.0	42.0	8.46	186671.0
5	Naruto	Shueisha	Shonen	250.0	72.0	8.09	449527.0
6	Demon Slayer: Kimetsu no Yaiba	Shueisha	Shonen	220.0	23.0	8.09	508330.0
7	Slam Dunk	Shueisha	Shonen	185.0	31.0	9.09	204691.0
8	Jujutsu Kaisen	Shueisha	Shonen	150.0	30.0	8.17	525383.0
9	Crayon Shin-chan	Futabasha	Seinen	148.0	65.0	7.97	6874.0

```
top10.iloc[:5, [0, 1, 2]]
```

	series	author_wiki	publisher
0	One Piece	Eiichiro Oda	Shueisha
1	Doraemon	Fujiko F. Fujio	Shogakukan
2	Golgo 13	Takao Saito , Saito Production	Shogakukan
3	Case Closed / Detective Conan	Gosho Aoyama	Shogakukan
4	Dragon Ball	Akira Toriyama	Shueisha

```
best_row = df_m["sales_m"].idxmax()
df_m.loc[best_row, ["series", "publisher", "sales_m", "score", "members", "primary_genre"]]
```

	0
series	One Piece
publisher	Shueisha
sales_m	600.0
score	9.2
members	710237.0
primary_genre	Action

Boolean indexing

Boolean indexing helps to find interesting groups: series that sold a lot and are rated highly, and "mismatches" - series with high sales but a below-median score (commercial success is not the same as acclaim), and the other way around. I used the medians of sales (32 million) and score (8.08) as thresholds; the third filter was written with query(), which is the same boolean filter written as a string, where @ refers to a Python variable.

Group	Series	Examples
High sales + high score	49	Berserk (9.46), Vagabond (9.27), One Piece (9.20), Slam Dunk (9.09)
High sales + low score	37	Golgo 13 (300M, 7.69), Crayon Shin-chan, Oishinbo, Astro Boy (6.77), Sazae-san (6.70)
Low sales + high score	34	Monster (9.16), Ashita no Joe, 20th Century Boys, Kaguya-sama, Death Note
Low sales + low score	45	

Note: the notebook printed 63 for the last group, but that number was calculated as "all the rest" and also contains the 18 series without a MAL score (a comparison with NaN is False). Among the 165 scored series the correct number is 45, and the four groups add up to 165. If sales and reception were unrelated, each quadrant would contain about a quarter of the series. The two "agreeing" quadrants contain 94 of 165 series (57%), which is a modest surplus over 50% and supports H1 only weakly; 71 series (43%) are mismatches.

Among the high-sales/low-score series there are several old titles, like Oishinbo, Golgo 13, Astro Boy and Captain Tsubasa, that probably do not go well with modern audiences. Also, series like Sazae-san and Crayon Shin-chan are directed at kids and families, so it is natural they are not rated as high. The low-sales/high-score series are mostly more mature titles; some, like Evangelion, are better known through their anime. Monster and 20th Century Boys are now considered classics, but were not as commercially successful as the action-packed shonen series.

Value counts

Publisher group	Series	Share
Shueisha	71	38.8%
Kodansha	44	24.0%
Shogakukan	32	17.5%
Other	11	6.0%
Akita Shoten	9	4.9%
Square Enix	7	3.8%
Hakusensha	6	3.3%
Nihon Bungeisha	3	1.6%

Shueisha is the biggest publisher in this dataset with about 39% of all best-selling titles, followed by Kodansha and Shogakukan; these three publishers together have about 80% of the series. Among demographics, Shonen is by far the most common one in the best-seller list: 110 series (60.1%), followed by Seinen (49, 26.8%), Shojo (19, 10.4%), Other (3, 1.6%) and Josei (2, 1.1%). This is expected, as almost all popular titles were initially targeted towards teenage boys.

The most frequent MAL genres in the best-sellers are Action (85 series), Award Winning (75), Comedy (69), Drama (63), Adventure (45), Fantasy (40), Romance (38),

Sports (27), Supernatural (23) and Sci-Fi (19). Action is expected, as titles like One Piece and Naruto are action-focused, while the high place of Award Winning shows that many best-sellers are also critically recognized.

```
df_m["genre_list"].explode().value_counts().head(10)
```

genre_list	count
Action	85
Award Winning	75
Comedy	69
Drama	63
Adventure	45
Fantasy	40
Romance	38
Sports	27
Supernatural	23
Sci-Fi	19

```
df_api["demographic"].value_counts(dropna=False)
```

demographic	count
Shonen	380
Seinen	311
NaN	222
Shojo	137
Josei	29
Kodomo	3

In contrast, the broader MAL popularity sample has a more balanced demographic distribution: among the series that have a demographic (860 of 1082), Shonen is 44%, Seinen 36%, Shojo 16% and Josei 3%, so Shojo and Josei are more prominent than in the best-seller list. It shows that the best-seller list is a special, commercially driven subset of manga.

```
med_sales, med_score = df_m["sales_m"].median(), df_m["score"].median()

hits = df_m[(df_m["sales_m"] >= med_sales) & (df_m["score"] >= med_score)]
hits[["series", "publisher", "sales_m", "score"]].sort_values("score", ascending=False).head(10)
```

	series	publisher	sales_m	score
33	Berserk	Hakusensha	70.0	9.46
27	Vagabond	Kodansha	82.0	9.27
0	One Piece	Shueisha	600.0	9.20
7	Slam Dunk	Shueisha	185.0	9.09
24	Fullmetal Alchemist	Square Enix	90.0	9.05
14	Kingdom	Shueisha	120.0	9.02
29	Haikyu!!	Shueisha	75.0	8.87
82	Frieren: Beyond Journey's End	Shogakukan	35.0	8.87
56	Nana	Shueisha	50.0	8.82
19	Hunter × Hunter	Shueisha	100.0	8.79

```
sales_without_acclaim = df_m[(df_m["sales_m"] >= med_sales) & (df_m["score"] < med_score)]
sales_without_acclaim[["series", "publisher", "sales_m", "score"]].sort_values("sales_m", ascending=False).head(10)
```

	series	publisher	sales_m	score
2	Golgo 13	Shogakukan	300.0	7.69
9	Crayon Shin-chan	Futabasha	148.0	7.97
11	Oishinbo	Shogakukan	135.0	7.38
12	Bleach	Shueisha	130.0	7.93
15	Astro Boy	Kobunsha	100.0	6.77
16	Baki the Grappler	Akita Shoten	100.0	7.74
21	My Hero Academia	Shueisha	100.0	7.94
23	Captain Tsubasa	Shueisha	90.0	7.55
25	Sazae-san	Kodansha	86.0	6.70
26	Kinnikuman	Shueisha	85.0	7.76

```
acclaim_without_sales = df_m.query("sales_m < @med_sales and score >= @med_score")
acclaim_without_sales[["series", "publisher", "sales_m", "score"]].sort_values("score", ascending=False).head(1
```

	series	publisher	sales_m	score
173	Monster	Shogakukan	20.0	9.16
124	Ashita no Joe	Kodansha	25.0	8.95
119	20th Century Boys	Shogakukan	26.4	8.93
142	Kaguya-sama: Love Is War	Shueisha	24.0	8.88
152	Made in Abyss	Takeshobo	22.0	8.71
120	Aoashi	Shogakukan	26.0	8.69
96	Death Note	Shueisha	30.0	8.68
100	Golden Kamuy	Shueisha	30.0	8.66
133	Neon Genesis Evangelion	Kadokawa	25.0	8.59
110	Chihayafuru	Kodansha	29.0	8.59

Aggregations: regular, multiple and named

A regular aggregation returns one statistic (the mean sales of all series is 54.13 million copies), a multiple one returns several statistics for one column, and a named aggregation (*agg(new_name=(column, function))*) computes different statistics for different columns and names the result columns at the same time. I used it to build the publisher summary.

Publisher	Series	Total sales (M)	Avg sales	Avg score	Avg members	Avg vol.
Shueisha	71	4,490.0	63.24	8.01	158,006	35.4
Kodansha	44	1,986.8	45.16	7.97	85,069	48.8
Shogakukan	32	1,933.9	60.43	8.07	48,813	54.8
Other	11	494.0	44.91	7.76	46,426	30.2
Akita Shoten	9	381.7	42.41	7.96	22,195	79.0
Square Enix	7	280.3	40.04	8.19	122,242	29.6
Hakusensha	6	224.5	37.42	8.10	166,795	56.0
Nihon Bungeisha	3	115.0	38.33	7.53	1,176	96.3

Demographic	Series	Avg sales	Median sales	Avg score	Avg members
Other	3	138.67	86.0	7.75	25,202
Shonen	110	60.41	38.0	8.00	120,747
Seinen	49	44.19	29.5	8.04	100,160
Josei	2	34.00	34.0	8.50	43,183
Shojo	19	32.21	28.0	7.91	39,048

These tables are the first answer to H2. Shueisha has the highest total sales (4,490 million, which is a result of having the most series), but the average sales per series are similar for Shueisha (63.2) and Shogakukan (60.4), and the median sales of every publisher are in the narrow range of 25-40 million: the means are pulled up by a few mega-hits (One Piece for Shueisha, Doraemon and Golgo 13 for Shogakukan). The average scores of the publishers are almost the same (7.8-8.2), while the average number of members differs a lot (from about 1,200 to 167,000). Among demographics, Shonen has a higher mean and median of sales than Seinen and Shojo, but a similar average score (8.00, 8.04 and 7.91). The "Other" group has only 3 series, so its high average is not reliable.

Transform

The *groupby().transform()* method returns a result of the same length as the original dataframe, so the group statistics can be added as a new column without a merge. I used it to compute the share of each series in the sales of its publisher, the rank of the series within its publisher and its score relative to its demographic (a z-score, which makes the scores of different demographics comparable). The table below shows how dependent each publisher is on its single best-selling series.

Publisher	Best-selling series	Share of publisher sales
Nihon Bungeisha	Minami no Teio	52.2%
Square Enix	Fullmetal Alchemist	32.1%
Hakusensha	Berserk	31.2%
Other	Crayon Shin-chan	30.0%
Akita Shoten	Baki the Grappler	26.2%
Shueisha	One Piece	13.4%
Kodansha	Attack on Titan	7.0%

A share of 0.5 or more means that one franchise brings half of the publisher's sales in this list. Shueisha and Kodansha are the least dependent on one title, because they have many series.

Merge / join

I attached the publisher-level summary back to every series with *merge* (the same can be done with df.join() when the key is in the index) and compared each series with the average of its publisher. For example, One Piece sells 9.5 times more than an average Shueisha series, Doraemon and Golgo 13 about 5 times more than an average Shogakukan series, and Case Closed 4.5 times more. Comparing the sales ratio with the score difference from the publisher average is a publisher-neutral way to look at the sales/reception relationship, because the publisher effect is removed from both. One Piece is also far above the average score of its publisher (+1.19), while Doraemon, Case Closed and Dragon Ball are only +0.2 to +0.45 above theirs, and Golgo 13 is below it (-0.38).

Pivot table

The pivot table below shows the average sales by publisher and demographic. A zero means that the publisher has no series in this demographic (fill_value=0).

Publisher	Josei	Other	Seinen	Shojo	Shonen
Akita Shoten	0	0	30.0	0	44.0
Hakusensha	0	0	49.8	31.2	0
Kodansha	34.0	86.0	38.4	31.7	49.1
Nihon Bungeisha	0	0	38.3	0	0
Other	0	0	49.7	0	39.2
Shogakukan	0	165.0	53.7	24.3	59.5
Shueisha	0	0	36.9	35.4	73.8
Square Enix	0	0	57.0	0	37.2

Kodansha is the most diverse publisher (all five demographics), while Shueisha is focused on Shonen, where it has the highest average sales among the large publishers (73.8 million). Most of the cells are empty, because a publisher usually specializes in one or two demographics: the publisher and the demographic are not independent, which matters when interpreting H2. I also built a second pivot table with two different aggregations (sum of sales and mean score) and margins. Its totals are lower than the real ones (8,307 against 9,906 million copies in total), apparently because the series without a MAL score are left out of that table, so I used the first pivot table for the conclusions.

Genres: exploding the list column

A manga usually has several genres, so to study the genre effect I used explode, which makes one row per manga-genre pair. It was done for both datasets: the merged one (genres vs. sales) and the broader API one (genres vs. reception). Only genres with enough series were kept (at least 5 in the merged data and 20 in the API data).

In the best-sellers, Adventure has the highest average sales (82.9 million, 45 series), followed by Mystery (69.1), Fantasy (66.7), Action (65.5) and Slice of Life (62.2). By the average score, Suspense (8.46, only 7 series), Horror (8.31, 7 series), Drama (8.17), Award Winning (8.16) and Fantasy (8.15) are on top: the more dramatic, darker series and the award winners are rated higher, but they do not have the highest sales. Ecchi has the lowest average sales (31.0) and one of the lowest scores (7.68), which is understandable for a genre that is not suitable for the mass market. In the larger API sample, Award Winning has the highest mean score (8.21, 160 series), followed by Boys Love (8.08) and Sports (8.07).

Melt

For comparing several metrics at once, the data has to be in the long format. I standardized four metrics (sales, score, members and favorites) into z-scores, so that they have the same scale, and used *melt* to turn the four columns into the metric / z_score pair. After melting, there are 4 rows for every series, which is the format that seaborn needs for grouped plots. Grouping by demographic and metric shows that Shonen series are slightly above average in sales (+0.10), members (+0.10) and favorites (+0.05) but exactly average in score (0.00), so the sales advantage of Shonen is not a reception advantage. Josei has the highest score z-score (+0.87) but below-average sales (-0.31), however it has only 2 series. Shojo is below average in sales, members and favorites (-0.30 to -0.41).

Task 8 - Testing the Hypotheses

H1: Do sales correlate with reception?

I computed two coefficients for every pair of variables: Pearson (on log-transformed variables for the skewed ones), which measures a linear relationship, and Spearman, which works with ranks and does not depend on the skewness and outliers. A p-value below 0.05 means that the relationship is unlikely to be random. The 18 series without a score were left out (n = 165).

X	Y	n	Pearson r	p	Spearman rho	p
sales_m	score	165	0.210	0.0067	0.187	0.0160
sales_m	scored_by	165	0.196	0.0114	0.158	0.0429
sales_m	members	165	0.185	0.0174	0.145	0.0632
sales_m	favorites	165	0.205	0.0083	0.170	0.0294
sales_per_volume_m	score	165	0.267	0.0005	0.256	0.0009
sales_per_volume_m	members	165	0.311	<0.0001	0.272	0.0004

Sales are weakly but significantly related to reception. For sales vs. score, Pearson r = 0.21 (p = 0.007) and Spearman rho = 0.19 (p = 0.016); scored_by and favorites give similar values. The relationship with members is borderline: the Spearman p-value of 0.063 is not significant at 0.05. So H1 is supported only weakly: r^2 is about 0.04, meaning the score explains only about 4% of the variation in sales. The correlations get stronger once the series length is removed: sales per volume vs. score gives r = 0.27 and vs. members r = 0.31 (both p < 0.001). A series that sells well per volume is better rated and more popular, while long series pile up total sales without being rated higher.

The full Spearman correlation matrix gives the following observations (it is also shown in the heatmap in Task 9):

- sales_m correlates much more with the series length (volumes_wiki, 0.44) than with the score (0.19), so length drives total sales more than reception does.
- volumes_wiki is almost unrelated to the score (-0.06): long series are not rated better or worse.
- scored_by, members and favorites correlate at 0.98-0.99, so they measure the same thing (audience size). Only one of them is needed in a model.
- popularity is a rank, so its -1.00 with members is expected and is not a real negative relationship.
- the score correlates 0.67-0.74 with the audience size: manga that more people read are also rated higher. This is a much stronger link than the one sales have with anything.

H2: Do the publisher, demographic and genre matter?

To measure how much a categorical variable explains, I used two things: eta squared, the share of the total variance that is explained by the groups (0 means nothing, 1 means

everything), and the Kruskal-Wallis test, a non-parametric test of whether the groups have the same distribution (the equivalent of one-way ANOVA that does not need normality). Groups with fewer than 3 series are dropped, since eta squared gets inflated when there are many groups with one or two members.

Group	Value	n	Groups	Eta squared	Kruskal H	p-value
publisher_grp	log_sales	183	8	0.0149	3.45	0.840
publisher_grp	score	164	7	0.0185	1.95	0.924
publisher_grp	log_members	183	8	0.1122	22.16	0.002
demographic	log_sales	181	4	0.0671	9.08	0.028
demographic	score	163	4	0.0072	0.47	0.925
demographic	log_members	181	4	0.0672	11.63	0.009
primary_genre	log_sales	179	6	0.0606	14.45	0.013
primary_genre	score	162	6	0.0602	8.67	0.123
primary_genre	log_members	179	6	0.1039	19.71	0.001

- **Publisher** explains almost none of the sales or the score (eta squared 0.015 and 0.019, $p > 0.8$). It does explain MAL popularity (0.11, $p = 0.002$), which probably reflects how widely the titles of a publisher are read internationally.
- **Demographic** has a small but significant effect on sales ($p = 0.028$) and on popularity ($p = 0.009$), and no effect on the score ($p = 0.93$).
- **Primary genre** is significant for sales (eta squared = 0.06, $p = 0.013$) and for popularity (0.10, $p = 0.001$), but not for the score ($p = 0.12$).
- All effects are small, with eta squared below 0.13, so most of the variation is not explained by the publisher, demographic or genre.

I repeated the analysis on the broader API dataset (all popular manga, groups with at least 10 series). The test statistic for the demographic could not be calculated, but its effect size is tiny.

Group	Value	n	Groups	Eta squared	p-value
demographic	score	838	5	0.0075	n/a
demographic	members	857	5	0.0211	n/a
primary_genre	score	1022	10	0.1048	< 0.0001
primary_genre	members	1040	10	0.0394	0.015
magazine	score	536	26	0.1191	< 0.0001
magazine	members	542	26	0.0831	0.003

- The primary genre explains 10% of the score variance and 4% of the popularity (both significant). Among all popular manga the genre has a real link with reception, unlike among the best-sellers.
- The magazine explains 12% of the score and 8% of the members, both significant. It has 26 groups, which inflates eta squared mechanically, and magazines overlap

strongly with the publisher and demographic, so I would not read it as an independent effect.

- The demographic has small effects (0.008 for the score, 0.021 for the members), so it matters little.

corr_table

	x	y	n	pearson_r	pearson_p	spearman_rho	spearman_p
0	sales_m	score	165	0.2104	0.0067	0.1873	0.0160
1	sales_m	scored_by	165	0.1964	0.0114	0.1579	0.0429
2	sales_m	members	165	0.1849	0.0174	0.1450	0.0632
3	sales_m	favorites	165	0.2048	0.0083	0.1696	0.0294
4	sales_per_volume_m	score	165	0.2670	0.0005	0.2561	0.0009
5	sales_per_volume_m	members	165	0.3106	0.0000	0.2719	0.0004

```
corr_cols = ["sales_m", "volumes_wiki", "sales_per_volume_m", "score", "scored_by", "members", "favorites", "popularity
corr_spearman = df_s[corr_cols].corr(method="spearman").round(2)
corr_spearman
```

	sales_m	volumes_wiki	sales_per_volume_m	score	scored_by	members	favorites	popularity
sales_m	1.00	0.44	0.49	0.19	0.16	0.14	0.17	-0.14
volumes_wiki	0.44	1.00	-0.48	-0.06	-0.08	-0.09	-0.07	0.09
sales_per_volume_m	0.49	-0.48	1.00	0.26	0.28	0.27	0.28	-0.27
score	0.19	-0.06	0.26	1.00	0.67	0.70	0.74	-0.70
scored_by	0.16	-0.08	0.28	0.67	1.00	0.99	0.98	-0.99
members	0.14	-0.09	0.27	0.70	0.99	1.00	0.99	-1.00
favorites	0.17	-0.07	0.28	0.74	0.98	0.99	1.00	-0.99
popularity	-0.14	0.09	-0.27	-0.70	-0.99	-1.00	-0.99	1.00

Далее: Создать код с переменной corr_spearman | New interactive sheet

	group	value	n	n_groups	eta_squared	kruskal_H	p_value
0	publisher_grp	log_sales	183	8	0.0149	3.4521	0.8403
1	publisher_grp	score	164	7	0.0185	1.9525	0.9240
2	publisher_grp	log_members	183	8	0.1122	22.1635	0.0024
3	demographic	log_sales	181	4	0.0671	9.0782	0.0283
4	demographic	score	163	4	0.0072	0.4702	0.9254
5	demographic	log_members	181	4	0.0672	11.6339	0.0087
6	primary_genre	log_sales	179	6	0.0606	14.4472	0.0130
7	primary_genre	score	162	6	0.0602	8.6744	0.1228
8	primary_genre	log_members	179	6	0.1039	19.7148	0.0014

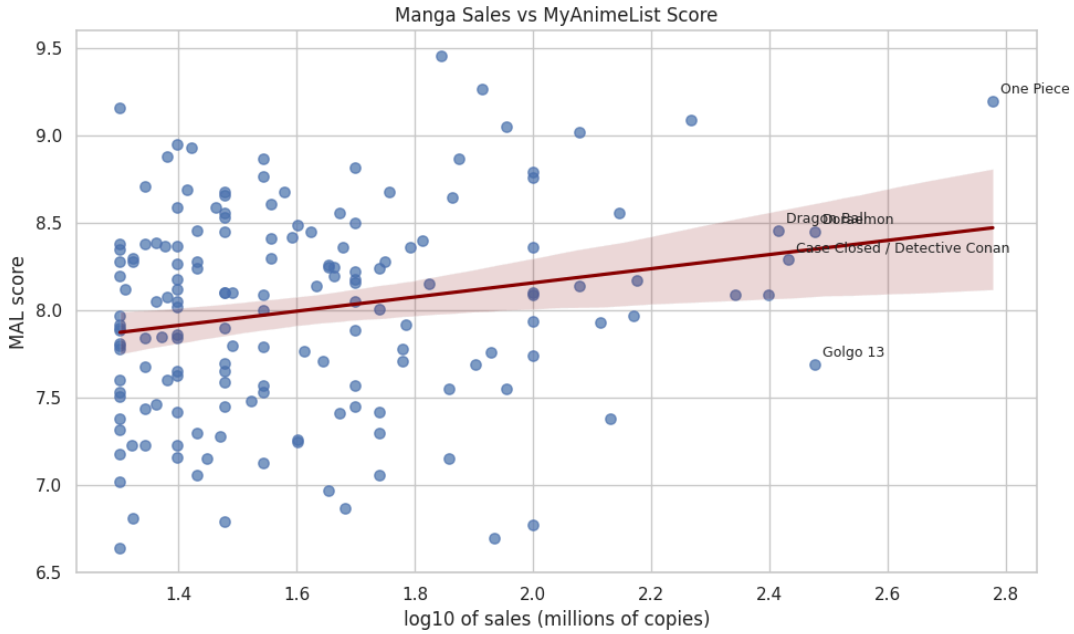
```
effects.pivot_table(index="group", columns="value", values="eta_squared")
```

value	log_members	log_sales	score
group			
demographic	0.0672	0.0671	0.0072
primary_genre	0.1039	0.0606	0.0602
publisher_grp	0.1122	0.0149	0.0185

Task 9 - Visualizations

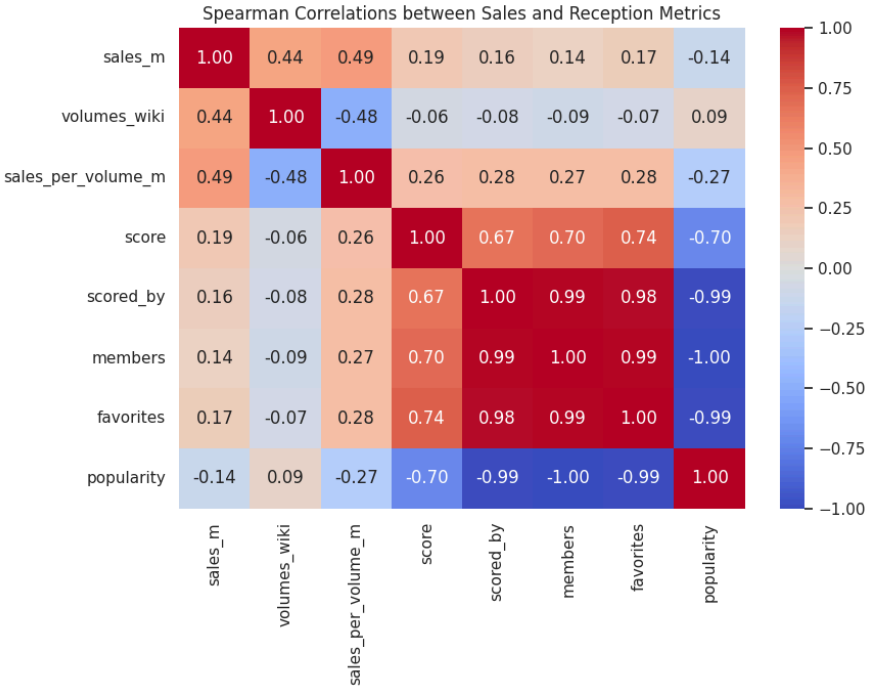
1. Sales vs MAL score (H1)

The main plot of the project: the x-axis is on a log scale because of the skewness of sales. The line is a linear regression with a confidence band, and the five best-selling series are labeled.



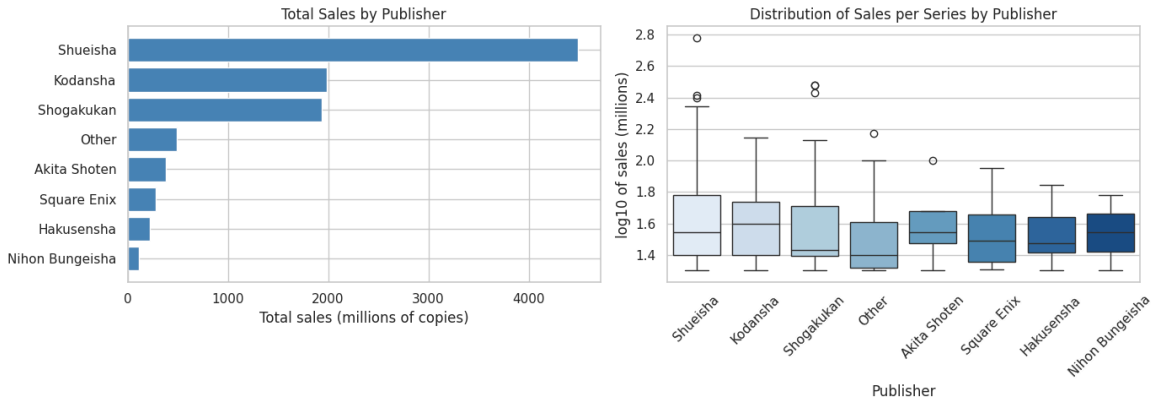
The points form a wide cloud and the regression line is only gently upward, from a score of about 7.9 for series with 20 million copies to about 8.5 for the biggest sellers. The confidence band becomes wider on the right side, because there are few points there. The best-sellers are rated well (One Piece has 9.20, and Dragon Ball, Doraemon and Case Closed are around 8.3-8.5), but a higher score does not mean more copies sold: Golgo 13 sold 300 million copies with a score of only 7.69, while series with sales of about 20 million have scores from 6.6 to 9.2. The left edge of the plot is at 20 million, because this is the lower limit of the Wikipedia list. This supports the weak positive correlation of H1.

2. Spearman correlation heatmap



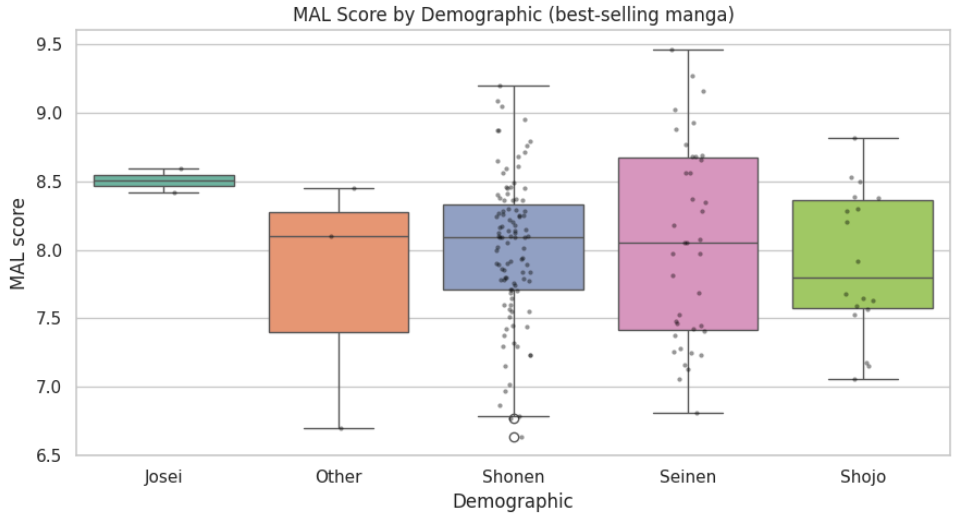
The heatmap puts the whole correlation matrix in one picture. The block of very strong correlations between members, favorites and scored_by (0.98-0.99) is expected, since they all measure the size of the audience. What matters for H1 is the sales_m row and the score column: sales correlate with the score at only 0.19, but with the series length at 0.44 and with the sales per volume at 0.49 (the last two are partly related by definition). The score has a strong link with the audience size (0.67-0.74).

3. Sales by publisher



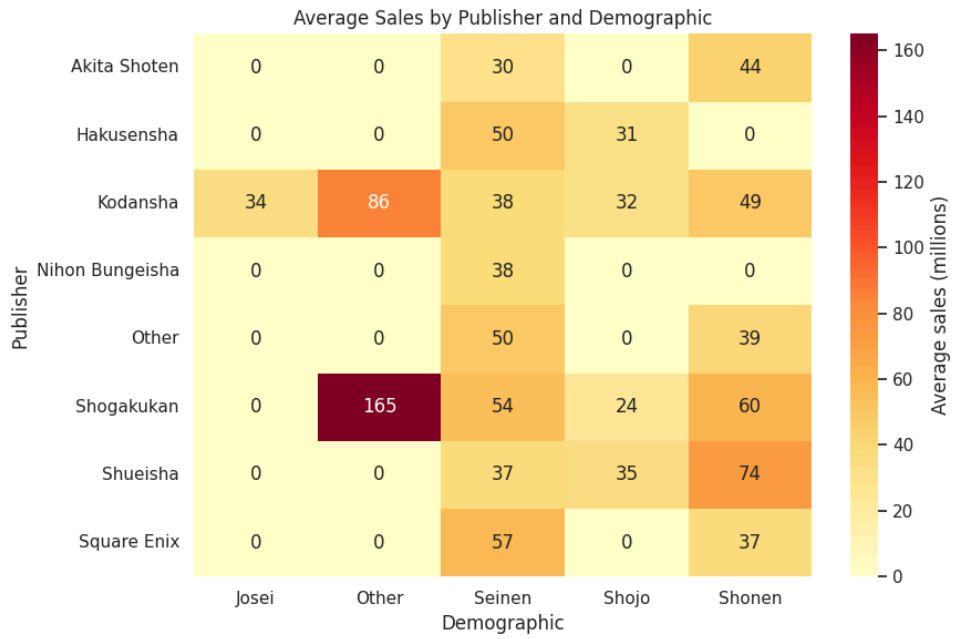
The bar chart shows the total sales, which also depends on the number of series: Shueisha has about 4,490 million copies, more than twice of Kodansha (1,987) and Shogakukan (1,934). The boxplot shows the distribution of sales per series, and here the publishers look much more similar: the medians are all close to each other (about 25-40 million, log10 of 1.4-1.6) and the boxes overlap. Shueisha and Shogakukan have high outliers (One Piece, Doraemon and Golgo 13), which explains why their mean sales are higher than the median. So the publisher's size means more titles, not more sales per title.

4. Score by demographic



Shonen, Seinen and the "Other" group have almost the same median score (about 8.05-8.1), while Shojo is a little lower (about 7.8). Shonen is the most common demography, so naturally it has a wide spread of scores with many points, including the lowest outliers (about 6.6). Seinen has the widest box (about 7.4-8.7) and also contains the highest-rated series (Berserk, 9.46). Josei has a high and compact score (about 8.5), but it has only 2 series, so this narrow range does not mean much. This agrees with the Kruskal-Wallis result, where the demographic does not explain the score ($p = 0.93$).

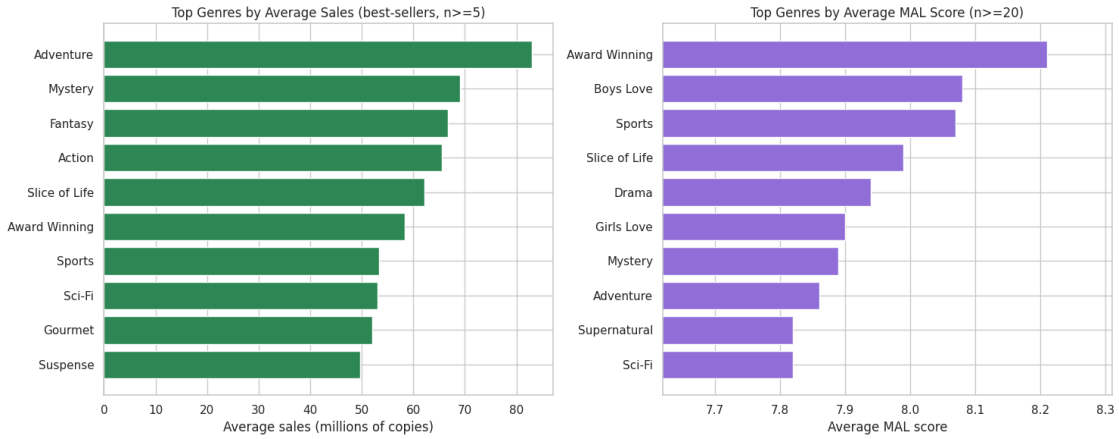
5. Publisher x demographic heatmap



This is the visual version of the pivot table. The zero cells show the specialization of the publishers (no series in that demographic), and the dark cells mark the combinations with the highest average sales. The darkest cell, Shogakukan x Other (165 million), is based on only two series (Doraemon and one smaller title), so it is not representative. Among the

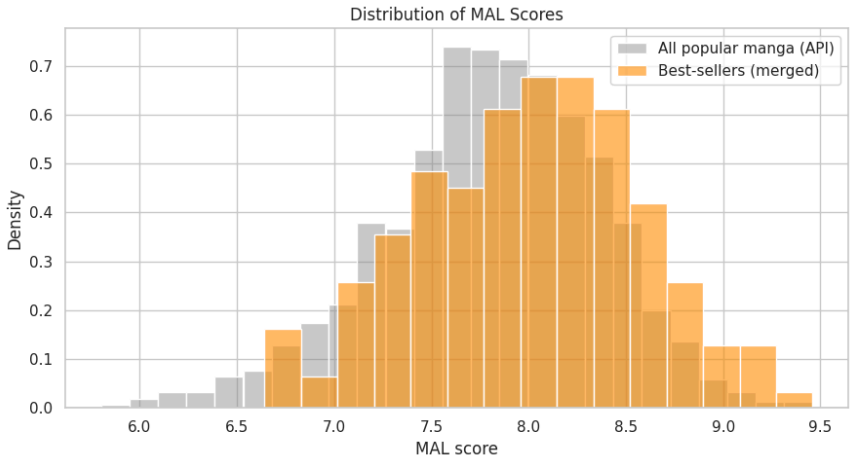
well-populated combinations, Shueisha x Shonen (74 million) and Shogakukan x Shonen (60 million) are the strongest.

6. Genres: sales and reception



The left panel is about commerce (best-sellers, at least 5 series per genre) and the right panel is about perception (all popular manga, at least 20 series per genre). The two rankings are different: Adventure, Mystery, Fantasy and Action lead in sales, while Award Winning, Boys Love and Sports lead in the score. Six genres appear in both top-10 lists (Adventure, Mystery, Slice of Life, Award Winning, Sports and Sci-Fi), but in a different order: Adventure is the top genre by sales and only 8th by score, Award Winning is 1st by score and 6th by sales, and Action and Fantasy are in the top 4 by sales but not in the top 10 by score. So a genre that readers rate highly is not necessarily the one that sells the most copies, which is another point against a strong H1. Note that the x-axis of the right panel starts at about 7.6, so the differences between genres look larger than they are (8.21 vs 7.82).

7. Score distribution: all popular manga vs best-sellers



The orange histogram (best-sellers) is shifted to the right compared with the grey one (all popular manga): the best-sellers have more mass in the 8.0-8.7 range and above 8.8, and almost no series below 6.7, while the grey histogram has a long left tail down to about 5.8. So best-selling manga are on average rated higher than a typical popular manga. This is a different question than the correlation within the best-sellers: it asks whether commercial

success in general goes together with a higher rating. The answer seems to be yes, but only a minimum level of quality is needed - once a series is in the best-seller list, a higher score does not mean much higher sales. The comparison is still affected by the selection: both groups consist of popular series, and the best-sellers are more likely to be rated by mainstream readers.

Task 10 - Conclusions

Hypotheses

H1 (sales are positively correlated with reception) is supported only weakly. All the correlations between sales and the reception metrics are positive and mostly significant, but small (r = 0.18-0.21, Spearman rho = 0.14-0.19), and the score explains only about 4% of the variation in sales. The relationship is stronger for sales per volume (r = 0.27-0.31), while the total sales mostly depend on the length of the series (rho = 0.44).

H2 (the publisher and the demographic/genre explain a significant share of variation) is mostly not supported. The publisher explains almost nothing of the sales and the score (eta squared 0.015 and 0.019), and the demographic and the genre explain only about 6-7% of the sales. The only larger effects are on the MAL popularity (publisher 11%, genre 10%).

Five findings

Finding 1: Most of the best-sellers can be linked to MAL, but only with a careful matching

183 of 201 best-selling series (91%) were linked to a MAL record. Only 96 of them were found by an exact title match; 68 more needed the search endpoint, and the relaxed matching needed manual checking, in which 8 doubtful matches were removed. Without the search and the manual review, the merged dataset would have been about half of its size or contained false matches.

Finding 2: Commercial success and reader acclaim are only weakly connected

The correlation between sales and the MAL score is r = 0.21 (rho = 0.19). Among the 165 scored series, 94 (57%) are either high in both sales and score or low in both, and 71 (43%) are mismatches. Golgo 13 (300 million copies, score 7.69) and Astro Boy (100 million, 6.77) sold a lot without acclaim, while Monster (9.16) and 20th Century Boys (8.93) are acclaimed but sold less than the median series. Series length matters more for sales (rho = 0.44) than the score (0.19).

Finding 3: Sales are concentrated in a few mega-hits, and the publisher does not explain the sales of a series

Shueisha has 39% of the best-selling series and about 45% of their total sales (4,490 of 9,906 million copies), but the median series of every publisher sells 25-40 million copies. The high means come from a few titles: One Piece (600 million) alone brings 13% of the Shueisha total, and Minami no Teio brings 52% of the total of Nihon Bungeisha. The publisher explains only 1.5% of the variance of sales (p = 0.84).

Finding 4: Shonen dominates the best-sellers, but not the reception

Shonen makes up 60% of the best-sellers, against 44% among the labeled series in the broader MAL sample, and it has higher average sales than Seinen and Shojo (60.4 vs 44.2 and 32.2 million). However, its average score (8.00) is equal to the overall average, so its advantage is commercial and not related to the reception. The demographic explains only 6.7% of the sales and 0.7% of the score.

Finding 5: Genre matters a little, and it affects sales and perception differently

Adventure has the highest mean sales (82.9 million) and Action, Fantasy and Mystery follow, while the highest scores belong to Award Winning, Suspense, Horror and Drama, so the genre rankings by sales and by score are different. The genre is the only variable that is significant for sales (eta squared 0.06, $p = 0.013$), and in the large API sample it explains 10% of the score variance, but all these effects are small.

Limitations

- The sales analysis is based on 183 best-sellers only, so the sample is restricted (all of them sold at least 20 million copies and have good scores). This range restriction makes the correlations weaker than they would be in the whole market.
- The Wikipedia sales are approximate and rounded (20, 25, 30 million and so on), come from different years and cover the whole run of a series, so long and old series have an advantage.
- The MAL score is an average of user ratings from an international online community, not a professional critic opinion, and it can be different from the opinion of the Japanese readers who buy most of the copies.
- The matching was done by titles, so some errors are possible even after the manual check, and 18 best-sellers have no MAL partner.
- I used the Jikan Edge backup API instead of the official Jikan API because of the outage, and the SSL certificate check was disabled (verify=False). 5 detailed records and 3 extra records could not be downloaded because of server errors.

Comparison of the two data collection methods

Aspect	API (Jikan / MyAnimeList)	Web scraping (Wikipedia)
Data format	Structured JSON	HTML tables with footnotes and merged cells
Size obtained	1082 manga, 22 columns	201 series, 13 columns
Main content	Readers' perception, genres, popularity	Sales, publishers, volumes
Parsing effort	json_normalize, explode of nested lists	Own table parser for rowspan/colspan, regex cleaning of text
Collection effort	1000 list + 995 detail + 94 extra requests, rate limit, retries	One request
Reliability	Outage during the project, a backup API was needed	Stable, but depends on the page layout
Missing values	Many (chapters, volumes, demographic)	None in the scraped columns
Joining	Has an ID (mal_id)	No ID: matched by titles

The two sources complement each other: the API has the reception data but no sales, and Wikipedia has the sales and publishers but no reception. Joining them was the hardest part of the project, because there is no common key, and the quality of the matching directly affects every result in the analysis.